

LAPORAN PRATIKUM STRUKTUR DATA

LAPORAN PRAKTIKUM PEKAN 4

Disusun Oleh:

Nama: Muhammad Aufa Rafiki

NIM: 2511531012

Dosen Pengampu: Dr. Wahyudi S.T.M.T

Asisten Praktikum: M. Galid



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Puji syukur penulis panjatkan kehadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan Laporan Praktikum Struktur Data ini dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas praktikum pada mata kuliah Struktur Data.

Laporan ini membahas mengenai implementasi struktur data Queue (antrian) dengan menggunakan beberapa metode, yaitu menggunakan array dan LinkedList. Selain itu, dibahas pula operasi dasar pada queue seperti enqueue, dequeue, serta pengembangan seperti iterasi dan pembalikan data (reverse). Melalui praktikum ini, penulis memperoleh pemahaman mengenai konsep queue serta penerapannya dalam pemrograman Java.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan untuk perbaikan di masa yang akan datang.

Akhir kata, semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca dalam memahami struktur data queue.

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Pratikum	1
1.3 Manfaat Pratikum	1
BAB II PEMBAHASAN	2
2.1 Dasar Teori	2
2.2 QueueArray_2511531012	2
2.3 QueueArrayDriveer_2511531012	4
2.4 QueueLinkedList_2511531012	5
2.5 ReverseData_2511531012.....	6
2.6 IterasiQueue_2511531012.....	7
BAB III KESIMPULAN	8
DAFTAR PUSTAKA	9
Tugas	10

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan cara untuk menyimpan dan mengelola data agar dapat digunakan secara efisien. Salah satu struktur data yang sering digunakan adalah Queue (antrian).

Queue menggunakan prinsip FIFO (First In First Out), yaitu data yang pertama masuk akan menjadi data pertama yang keluar. Struktur ini banyak digunakan dalam kehidupan nyata seperti antrian di bank, sistem printer, dan proses penjadwalan.

1.2 Tujuan Pratikum

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut:

1. Memahami konsep dasar Queue
2. Mengimplementasikan Queue menggunakan array
3. Menggunakan Queue dengan LinkedList
4. Mempelajari operasi enqueue dan dequeue
5. Menerapkan iterasi dan reverse pada Queue

1.3 Manfaat Praktikum

1. Menambah pemahaman tentang struktur data Queue
2. Melatih kemampuan pemrograman Java
3. Mengetahui perbedaan implementasi Queue
4. Dapat menerapkan konsep Queue dalam kasus nyata

BAB II

PEMBAHASAN

2.1 Dasar teori

Queue (antrian) adalah struktur data linear yang menerapkan prinsip FIFO (First In First Out), yaitu elemen yang pertama masuk akan menjadi elemen pertama yang keluar.

Queue memiliki dua operasi utama, yaitu enqueue untuk menambahkan elemen ke bagian belakang dan dequeue untuk menghapus elemen dari bagian depan.

Selain itu, terdapat operasi pendukung seperti pengecekan kondisi kosong (isEmpty) dan penuh (isFull).

Dalam implementasinya, queue dapat menggunakan array atau linked list. Salah satu bentuk pengembangan dari queue adalah circular queue, yaitu antrian yang memungkinkan perputaran indeks sehingga ruang kosong dapat digunakan kembali secara efisien.

Struktur data queue banyak digunakan dalam berbagai sistem yang membutuhkan pengolahan data secara berurutan sesuai waktu kedatangan.

2.2 QueueArray_2511531012

```
1 package pekan4_2511531012;
2
3 public class QueueArray_2511531012 {
4     int front_1012, rear_1012, size_1012;
5     int capacity_1012;
6     int array[];
7
8     public QueueArray_2511531012(int capacity_1012) {
9         this.capacity_1012 = capacity_1012;
10        front_1012 = this.size_1012 = 0;
11        rear_1012 = capacity_1012 - 1;
12        array = new int[this.capacity_1012];
13    }
14
15    boolean isFull_1012(QueueArray_2511531012 queue) {
16        return (queue.size_1012 == queue.capacity_1012);
17    }
18
19    boolean isEmpty_1012(QueueArray_2511531012 queue) {
20        return (queue.size_1012 == 0);
21    }
22
23    void enqueue_1012(int item) {
24        if (isFull_1012(this))
25            return;
26        this.rear_1012 = (this.rear_1012 + 1) % this.capacity_1012;
27        this.array[this.rear_1012] = item;
28        this.size_1012 = this.size_1012 + 1;
29        System.out.println(item + "enqueue to queue");
30    }
31}
```

```

32  int dequeue_1012() {
33      if(isEmpty_1012(this))
34          return Integer.MIN_VALUE;
35      int item = this.array[this.front_1012];
36      this.front_1012= (this.front_1012 + 1) % this.capacity_1012;
37      this.size_1012= this.size_1012 - 1;
38      return item;
39  }
40  int front_1012() {
41      if(isEmpty_1012(this))
42          return Integer.MIN_VALUE;
43      return this.array[this.front_1012];
44  }
45  int rear_1012() {
46      if (isEmpty_1012(this))
47          return Integer.MIN_VALUE;
48      return this.array[this.rear_1012];
49  }
50
51  void display_1012() {
52      int i;
53      if (size_1012 == 0) {
54          System.out.printf("\nAntrian Kosong\n");
55          return;
56      }
57
58      for (i= front_1012; i< rear_1012; i++) {
59          System.out.printf("%d <-- ",array[i]);
60      }
61      return;
62  }
63
64 }
65

```

Penjelasan Program:

1. QueueArray_2511531012(int capacity_1012) →(konduktor) Menginisialisasi queue dengan kapasitas tertentu.
2. isFull_1012(QueueArray_2511531012 queue) → Mengecek apakah queue dalam kondisi penuh. Caranya dengan membandingkan jumlah elemen (size) dengan kapasitas (capacity), jika sama maka Queue penuh.
3. isEmpty_1012(QueueArray_2511531012 queue) → Mengecek apakah queue kosong. Caranya, Jika size == 0 maka queue kosong.
4. enqueue_1012(int item) → Menambahkan elemen ke dalam queue (bagian belakang). Caranya: mengecek apakah queue penuh, jika tidak maka posisi rear digeser, lalu data dimasukkan ke array dan size ditambah.
5. dequeue_1012() → Menghapus elemen dari bagian depan queue. Caranya: mengecek apakah queue kosong, jika tidak maka ambil data di posisi front, lalu geser front dan size berkurang.
6. front_1012() → Mengambil elemen paling depan tanpa menghapusnya. Caranya: mengembalikan nilai array pada indeks front.
7. rear_1012() → Mengambil elemen paling belakang. Caranya: Mengembalikan nilai array pada indeks rear.
8. display_1012() → Menampilkan seluruh isi queue. Caranya: jika kosong maka tampilkan pesan, jika tidak maka menampilkan elemen dari front ke rear.

2.3 QueueArrayDriver_2511531012

```
1 package pekan4_2511531012;
2
3 public class QueueArrayDriver_2511531012 {
4
5     public static void main(String[] args) {
6         QueueArray_2511531012 queue_1012 = new QueueArray_2511531012(1000);
7         queue_1012.enqueue_1012(10);
8         queue_1012.enqueue_1012(20);
9         queue_1012.enqueue_1012(30);
10        queue_1012.enqueue_1012(40);
11
12        System.out.println("Item di depan " + queue_1012.front_1012());
13        System.out.println("Item paling belakang " + queue_1012.rear_1012());
14        System.out.println("Tampilan Queue");
15        queue_1012.display_1012();
16        System.out.println();
17
18        System.out.println(queue_1012.dequeue_1012() + " dihapus dari Queue");
19        System.out.println("item di depan : " + queue_1012.front_1012());
20        System.out.println("item di belakang : " + queue_1012.rear_1012());
21        System.out.println("Tampilan Queue setelah satu Data dihapus");
22        queue_1012.display_1012();
23    }
24 }
25
```

Penjelasan Program:

1. Membuat objek queue
2. Menambahkan beberapa data
3. Menampilkan:
 - Elemen depan → menggunakan front_1012()
 - Elemen belakang → menggunakan rear_1012()
4. Menampilkan isi queue → display_1012()
5. Menghapus satu elemen → dequeue_1012()
6. Menampilkan kondisi terbaru

Output:

```
10enqueue to queue
20enqueue to queue
30enqueue to queue
40enqueue to queue
Item di depan 10
Item paling belakang 40
Tampilan Queue
10 <-- 20 <-- 30 <--
10 dihapus dari Queue
item di depan : 20
item di belakang : 40
Tampilan Queue setelah satu Data dihapus
20 <-- 30 <--
```

2.4 QueueLinkedList_2511531012

```
1 package pekan4_2511531012;
2
3 import java.util.Queue;
4
5
6 public class QueueLinkedList_2511531012 {
7
8     public static void main(String[] args) {
9         Queue<Integer> q_1012 = new LinkedList<>();
10        for (int i=0; i<6; i++)
11            q_1012.add(i);
12        System.out.println("Elemen Antrian " + q_1012);
13
14        int hapus_1012 = q_1012.remove();
15        System.out.println("Hapus Elemen = " + hapus_1012);
16        System.out.println(q_1012);
17
18        int depan_1012 = q_1012.peek();
19        System.out.println("Kepala Antrian = " + depan_1012);
20
21        int banyak_1012 = q_1012.size();
22        System.out.println("Size Antrian = " + banyak_1012);
23
24    }
25
26 }
27
```

Penjelasan Program:

1. Queue<Integer> → mendeklarasikan queue dengan tipe data Integer
2. q_1012=new LinkedList<>() → membuat objek queue menggunakan LinkedList
3. add(i)→ Menambahkan elemen ke dalam queue dari i=0 sampai i<6
4. remove()→ Menghapus elemen dari depan queue.
5. peek() → Melihat elemen paling depan tanpa menghapusnya.
6. size() → Mengetahui jumlah elemen dalam queue.

Output:

```
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus Elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```


2.5 ReverseData_2511531012

```
1 package pekan4_2511531012;
2 import java.util.LinkedList;
3
4
5
6 public class ReverseData_2511531012 {
7
8     public static void main(String[] args) {
9         Queue<Integer> q_1012= new LinkedList<Integer>();
10        q_1012.add(1);
11        q_1012.add(2);
12        q_1012.add(3);
13        System.out.println("sebelum reverse" + q_1012);
14        Stack<Integer> s = new Stack<Integer>();
15        while (!q_1012.isEmpty()) {
16            s.push(q_1012.remove());
17        }
18        while (!s.isEmpty()) {
19            q_1012.add(s.pop());
20        }
21        System.out.println("sesudah reverse = " + q_1012);
22    }
23 }
24
25 }
26 }
```

Penjelasan program:

1. Queue<Integer> q_1012 = new LinkedList<Integer>(); → Membuat queue kosong dengan tipe data Integer.
2. add() → Menambahkan elemen ke dalam queue.
3. Menampilkan isi queue sebelum dilakukan reverse [1,2,3]
4. Stack<Integer> s = new Stack<Integer>(); → Membuat stack kosong untuk membantu proses pembalikan data.
5. !q_1012.isEmpty() → mengecek apakah queue masih ada isi.
6. s.push(q_1012.remove()) → memasukkan data ke stack yang mana mengambil data dari depan queue.
Hasil: [1,2,3] → [3,2,1]
7. while (!s.isEmpty()) { q_1012.add(s.pop()); } → Memindahkan Stack ke Queue, mengambil data dari stack (dari atas), lalu masukkan kembali ke queue.
Hasil : [3,2,1] → [3,2,1]
8. Menampilkan Queue Setelah Reverse [3,2,1]

Output:

```
sebelum reverse[1, 2, 3]
sesudah reverse = [3, 2, 1]
```

2.6 IterasiQueue_2511531012

```
1 package pekan4_2511531012;
2
3 import java.util.Iterator;
4
5
6
7 public class IterasiQueue_2511531012 {
8
9     public static void main(String[] args) {
10         Queue<String> q_1012 = new LinkedList<>();
11         q_1012.add("Praktikum");
12         q_1012.add("Struktur");
13         q_1012.add("Data");
14         q_1012.add("Dan");
15         q_1012.add("Algoritma");
16         Iterator<String> iterator = q_1012.iterator();
17         while (iterator.hasNext()) {
18             System.out.print(iterator.next()+ " ");
19         }
20
21     }
22 }
23
```

Penjelasan Program:

1. Queue<String> q_1012 = new LinkedList<>(); → Membuat queue dengan tipe data String.
2. q_1012.add() → method add() ini untuk menambahkan data/elemen ke dalam queue
3. Iterator<String> iterator = q_1012.iterator(); → method iterator() digunakan untuk mengambil objek iterator dari queue.
4. while (iterator.hasNext()) → Method hasNext() digunakan mengecek apakah masih ada elemen berikutnya.
5. Method next() → iterator.next() → digunakan untuk mengambil elemen berikutnya dari queue.

Output:

```
Praktikum Struktur Data Dan Algoritma
```

BAB III

KESIMPULAN

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Queue (antrian) merupakan salah satu struktur data linear yang menggunakan prinsip FIFO (First In First Out), yaitu data yang pertama kali masuk akan menjadi data pertama yang keluar.

Pada praktikum ini, antrian diimplementasikan menggunakan array dengan konsep circular queue, sehingga penggunaan memori menjadi lebih efisien karena indeks dapat kembali ke awal ketika mencapai batas maksimum array. Program yang dibuat mampu melakukan operasi dasar pada queue seperti enqueue (menambah data), dequeue (menghapus data), menampilkan isi antrian, serta melakukan reverse pada data.

Selain itu, penggunaan variabel seperti front, rear, size, dan capacity mempermudah dalam pengelolaan antrian, terutama dalam pengecekan kondisi penuh dan kosong. Dengan adanya menu interaktif, program juga menjadi lebih mudah digunakan dan sesuai dengan studi kasus antrian loket pelayanan.

Dengan demikian, praktikum ini memberikan pemahaman yang baik mengenai konsep queue serta penerapannya dalam permasalahan nyata

DAFTAR PUSTAKA

- [1] I. Liem, *Draft Diktat Struktur Data*, Edisi 2008. Bandung, Indonesia: Program Studi Teknik Informatika, Institut Teknologi Bandung, 2008.

Tugas

AntrianLoket_2511531012

```
1 package pekan4_2511531012;
2
3 public class AntrianLoket_2511531012 {
4     int front_1012, rear_1012, size_1012;
5     int capacity_1012;
6     String queue_1012[];
7
8     public AntrianLoket_2511531012(int capacity_1012) {
9         this.capacity_1012 = capacity_1012;
10        front_1012 = this.size_1012 = 0;
11        rear_1012 = capacity_1012 - 1;
12        queue_1012 = new String[this.capacity_1012];
13    }
14
15    boolean isFull_1012(AntrianLoket_2511531012 queue) {
16        return (queue.size_1012 == queue.capacity_1012);
17    }
18
19    boolean isEmpty_1012(AntrianLoket_2511531012 queue) {
20        return (queue.size_1012 == 0);
21    }
22
23    void enqueue_1012(String nama_1012) {
24        if (isFull_1012(this)) {
25            System.out.println("Antrian penuh");
26            return;
27        }
28        this.rear_1012 = (this.rear_1012 + 1) % this.capacity_1012;
29        this.queue_1012[this.rear_1012] = nama_1012;
30        this.size_1012 = this.size_1012 + 1;
31        System.out.println("Data berhasil ditambahkan ke antrian");
32    }
33
34    void dequeue_1012() {
35        if (isEmpty_1012(this)) {
36            System.out.println("Antrian kosong");
37            return;
38        }
39        String nama_1012 = this.queue_1012[this.front_1012];
40        this.front_1012 = (this.front_1012 + 1) % this.capacity_1012;
41        this.size_1012 = this.size_1012 - 1;
42        System.out.println(nama_1012 + " telah dilayani");
43    }
44
45    void display_1012() {
46        if (isEmpty_1012(this)) {
47            System.out.println("Antrian kosong");
48            return;
49        }
50
51        System.out.println("Isi antrian:");
52        int i = front_1012;
53        int nomor = 1;
54
55        for (int count = 0; count < size_1012; count++) {
56            System.out.println(nomor + ". " + queue_1012[i]);
57            i = (i + 1) % capacity_1012;
58            nomor++;
59        }
60    }
61}
```

```

62● void reverse_1012() {
63●     if (isEmpty_1012(this)) {
64         System.out.println("Antrian kosong");
65         return;
66     }
67
68     String temp_1012[] = new String[capacity_1012];
69     int i = front_1012;
70
71●     for (int j = 0; j < size_1012; j++) {
72         temp_1012[j] = queue_1012[i];
73         i = (i + 1) % capacity_1012;
74     }
75
76     int k = size_1012 - 1;
77     i = front_1012;
78
79●     for (int j = 0; j < size_1012; j++) {
80         queue_1012[i] = temp_1012[k];
81         i = (i + 1) % capacity_1012;
82         k--;
83     }
84
85     System.out.println("Antrian berhasil dibalik");
86 }
87 }

```

Penjelasan Program:

1. isFull_1012(AntrianLoket_2511531012 queue)

Method ini berfungsi untuk mengecek apakah antrian dalam kondisi penuh. Jika nilai size_1012 sama dengan capacity_1012, maka antrian dinyatakan penuh dan tidak bisa menambahkan data baru.

2. isEmpty_1012(AntrianLoket_2511531012 queue)

Method ini berfungsi untuk mengecek apakah antrian kosong. Jika nilai size_1012 bernilai 0, maka tidak ada data dalam antrian.

3. enqueue_1012(String nama_1012)

Method ini digunakan untuk menambahkan data (nama pelanggan) ke dalam antrian.

Proses yang dilakukan:

- Mengecek apakah antrian penuh
- Menggeser posisi rear dengan rumus circular $(rear + 1) \% capacity$
- Menyimpan data ke dalam array
- Menambahkan nilai size
- Menampilkan pesan bahwa data berhasil ditambahkan

4. dequeue_1012()

Method ini digunakan untuk menghapus data dari bagian depan antrian.

Proses yang dilakukan:

- Mengecek apakah antrian kosong.
- Mengambil data pada posisi front
- Menggeser posisi front dengan rumus circular
- Mengurangi nilai size
- Menampilkan data yang telah dilayani

5. display_1012()

Method ini digunakan untuk menampilkan seluruh isi antrian.

Proses:

- Mengecek apakah antrian kosong
- Menampilkan data mulai dari front sebanyak size
- Menggunakan perulangan dengan konsep circular agar tetap berjalan meskipun indeks kembali ke awal array

6. everse_1012()

Method ini digunakan untuk membalik urutan antrian.

Proses:

- Menyimpan data ke array sementara
- Membalik urutan data
- Mengembalikan data ke array utama
- Menampilkan pesan bahwa antrian berhasil dibalik

AntrianLoketDriver_2511531012

```

1 package pekan4_2511531012;
2
3 import java.util.Scanner;
4
5 public class AntrianLoketDriver_2511531012 {
6     public static void main(String[] args) {
7         Scanner input_1012 = new Scanner(System.in);
8         AntrianLoket_2511531012 antrian_1012 = new AntrianLoket_2511531012(10);
9
10        int pilih_1012;
11
12        do {
13            System.out.println("\n== PROGRAM ANTRIAN LOKET ==");
14            System.out.println("1. Tambah Antrian");
15            System.out.println("2. Hapus Antrian");
16            System.out.println("3. Tampilkan Antrian");
17            System.out.println("4. Reverse");
18            System.out.println("5. Keluar");
19            System.out.print("Pilih menu: ");
20            pilih_1012 = input_1012.nextInt();
21            input_1012.nextLine();
22
23            switch (pilih_1012) {
24                case 1:
25                    System.out.print("Masukkan nama pelanggan: ");
26                    String nama_1012 = input_1012.nextLine();
27                    antrian_1012.enqueue_1012(nama_1012);
28                    break;
29
30                case 2:
31                    antrian_1012.dequeue_1012();
32                    break;
33
34                case 3:
35                    antrian_1012.display_1012();
36                    break;
37
38                case 4:
39                    antrian_1012.reverse_1012();
40                    antrian_1012.display_1012();
41                    break;
42
43                case 5:
44                    System.out.println("Program selesai");
45                    break;
46
47                default:
48                    System.out.println("Pilihan tidak valid");
49            }
50        } while (pilih_1012 != 5);
51    }
52 }
53 }

```

Penjelasannya:

1. Scanner input_1012 → Digunakan untuk menerima input dari pengguna.
2. AntrianLoket_1012 antrian_1012 → Membuat objek antrian dengan kapasitas tertentu.
3. Variabel pilih_1012 → Digunakan untuk menyimpan pilihan menu dari pengguna.
4. Struktur do-while → Digunakan untuk menjalankan program menu secara

berulang sampai pengguna memilih keluar.

5. Struktur switch-case → Digunakan untuk menjalankan fitur sesuai pilihan menu:

- Menu 1 → Menambahkan data ke antrian
- Menu 2 → Menghapus data dari antrian
- Menu 3 → Menampilkan isi antrian
- Menu 4 → Membalik antrian
- Menu 5 → Keluar dari program

Output:

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: aufa
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: dafa
Data berhasil ditambahkan ke antrian
```

```
=== PROGRAM ANTRIAN LOKET ===
```

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

Pilih menu: 3

Isi antrian:

1. aufa
2. dafa

```
=== PROGRAM ANTRIAN LOKET ===
```

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

Pilih menu: 4

Antrian berhasil dibalik

Isi antrian:

1. dafa
2. aufa

```
=== PROGRAM ANTRIAN LOKET ===
```

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

Pilih menu: 2

dafa telah dilayani

```
=== PROGRAM ANTRIAN LOKET ===
```

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

Pilih menu: 3

Isi antrian:

1. aufa

